

The backend: everything I built, explained

This document explains every part of the new backend in plain words: what a visitor sees, what happens with payments, the admin pages, the database, the emails, the timed jobs, the safety rules, how it goes onto the server, how it was tested, and what is still missing.

~7,400

lines of new code

88 / 88

automatic checks pass

13

database tables

5

admin roles

Contents

1. What a "backend" is, and the three new folders
2. A professional registers and pays
3. A student registers
4. Psoola: how payments are connected
5. Every payment situation, and what happens
6. Website pages I changed or added
7. The addresses the pages talk to (API)
8. The admin: every page
9. Roles: who can see what
10. The database: all 13 tables
11. The emails
12. The PDF ticket and the QR code
13. Timed jobs (automatic work)
14. Settings you control
15. Safety and security
16. Putting it on the server
17. How it was tested (all 88 checks)
18. The security review and the fixes
19. What is not finished yet
20. Every file I made

1 What a "backend" is, and the three new folders

Until now the website was only pages: the registration form could not save anything, so it sent people to WhatsApp. The **backend** is the part that runs on the server. It receives the form, saves it in a database, talks to Psoola for the money, makes tickets, sends emails, and gives you the admin to see and manage everything.

api/

what the website pages call:
register, pay, payment status,
QR image, sponsor, webhook
public, 10 small files

admin/

the pages YOU use at /admin/:
dashboard, registrations, students,
payments, workshops, sponsors...
sign-in + phone code

backend/

the actual brain: rules, payments,
tickets, emails, database, secrets
+ timed jobs, tools, tests
NOT reachable from the internet

Language: PHP 8.1 · Database: MySQL/MariaDB

what every cPanel hosting has · no framework · one library (TCPDF, for PDF + QR)

MySQL database (13 tables)

every registration, payment, ticket, email

2 A professional registers and pays

- 1 They open **register.html**. The page first asks the server: *is registration open, what are the prices, which lunch days are left?* If closed, they see "Registration opens soon" instead of the form.
- 2 They fill in the 3 steps (details, lunch, payment method: Visa, MasterCard, FIB or FastPay) and press the button.
- 3 The server checks everything again (it never trusts the page): name, phone, email, allowed choices, capacity, spam trap, too many tries.

- 4 It saves the registration with a reference like **ISM26-CB4TNZ** and **works out the price itself** (ticket + chosen lunch days) from the prices in the admin.
- 5 It asks Psoola to create a payment for exactly that amount and sends the visitor to Psoola's payment page.
- 6 The visitor pays. Psoola sends them back to our new **payment.html**, which shows "Checking your payment...".
- 7 Psoola also sends our server a message (webhook). Our server does **not** believe the message: it asks Psoola directly "is payment X paid, and how much?"
- 8 Only if Psoola says **paid, the right amount, the right currency**: the registration becomes paid and a **ticket** is made (T26-00001, T26-00002...).
- 9 The page changes to **"You are in! Welcome to iSmile 2026"** with the QR code, and the ticket email with the PDF goes out.

3 A student registers

- 1 Same form, but they choose "dental student", add their university, and upload a **photo of their student ID**. (Choosing "student" always gives the student ticket, whatever else they pick.)
- 2 The photo is checked: must really be an image (a renamed program is refused), is re-saved clean, and stored in a **private folder** under a random name.
- 3 No payment yet. They get the email "we are checking your student ID".
- 4 Someone with the Registration role opens **Student approvals** in the admin, sees the photo large, and presses **Approve** or **Reject** with a reason.
- 5 **Approved** → they get a "Pay now" email with a personal link → they pay the student price → ticket.
- 6 **Rejected** → they get the reason and a link to **student-id.html** to send a clearer photo → it comes back to the approvals list.

Why approve first? So nobody pays and then needs a refund because their ID was not valid. Refunds are the messiest part of any payment system.

4 Psoola: how payments are connected

There are exactly three connections with Psoola. All the code around them is finished; only the three places where we speak Psoola's "language" wait for their documents.

	Connection	What it does	State
A	Create a payment	Our server tells Psoola: "payment for ISM26-..., amount X IQD" and gets back the address of Psoola's payment page.	needs Psoola docs
B	Webhook (Psoola → us)	Psoola tells us "payment X changed". Our server checks Psoola's signature on the message, writes it in the log, and then uses C. The message itself never makes a ticket.	needs Psoola docs
C	Check a payment (us → Psoola)	Our server asks Psoola: "what is the status and amount of payment X?" This answer is the only thing that can make a ticket.	needs Psoola docs

Until Psoola's documents arrive, a pretend Psoola runs on the test site (`api/fake-psoola.php`). It shows a page where the tester chooses what happens: **pay**, **fail**, **pay a smaller amount**, **pay but lose the webhook**, or **close the page**. That is how every situation below was tested. The pretend Psoola refuses to run on the real address.

5 Every payment situation, and what happens

Situation	What the system does
Paid normally	Ticket made, ticket email sent, status page shows the QR.
Card declined / payment failed	Stays unpaid, no ticket. One "Pay now" email with a personal link so they can try again without filling the form again.
Declined, then paid on the same Psoola page	The late "paid" is still accepted and the ticket is made; the money is never lost. fixed after review
Psoola confirms a different amount	NOT paid. Marked mismatch in red for Finance, alert email, no ticket.
Paid twice (two tabs)	Only one ticket. The second payment is marked duplicate in red: Finance refunds it.
Paid, but Psoola's message is lost	Every 5 minutes the server asks Psoola about all waiting payments, finds it paid, makes the ticket.
The same message arrives twice	Nothing changes the second time; still exactly one ticket.
A fake "paid" message from a hacker	Refused (wrong signature), logged as REJECTED. After 3 in one hour, an alert email.
Visitor closed the page before paying	Stays unpaid. After 24 hours the attempt closes; they get the "Pay now" link. After 3 days unpaid: one reminder.
Clicked "Pay" many times	Within 10 minutes the same payment is reused, so no pile of payments at Psoola. fixed after review
Psoola is down	Registration stays saved; "Pay now" email, at most one a day. fixed after review
Last places, many paying at once	A place is held for 1 hour for the person paying; others see "full". If it ever goes over anyway, the ticket is made and the Owner is alerted. fixed after review
Registration switched off (emergency)	No new payment can start from anywhere, including "Pay now" links. fixed after review
Price still 0	Cannot be paid, and registration cannot be opened until prices are set.

6 Website pages I changed or added

Page	What changed
register.html	Asks the server if registration is open and shows the real prices. Sends the form to the server (not WhatsApp), then to Psoola. "Registration opens soon" panel while closed (your own message possible, in 3 languages). Hidden spam trap. Clear messages for every error, in the visitor's language.
payment.html new	Where people land after paying: "Checking..." → "You are in!" with QR and ticket number, or "Payment did not go through" with Try again , or "We are checking your student ID".
student-id.html new	A rejected student sends a clearer photo from their personal link.

sponsor.html	The request is saved on the server with a reference (SPN26-...); the company and your sponsor team get emails. If the server cannot be reached, the old WhatsApp/email step appears, so no request is lost.
Language files	About 47 new texts in each of English, Arabic and Kurdish (payment page, errors, closed message).

7 The addresses the pages talk to (API)

Address	Used by	What it does
api/config.php	register.html	Open or closed, prices, which lunch days are left.
api/register.php	register.html	Saves the registration (+ student photo); starts the payment for professionals.
api/pay.php	"Pay now" emails, Try again	Personal, expiring link that starts a new payment without the form.
api/status.php	payment.html	Is it paid? Asks Psoola if still waiting (at most every 10 seconds).
api/qr.php	payment.html, emails	The ticket QR as an image, personal link only.
api/student-photo.php	student-id.html	New ID photo after a rejection.
api/sponsor.php	sponsor.html	Saves a sponsor or booth request.
api/webhook.php	Psoola	Psoola's "payment changed" messages. This is the address we give Psoola.
api/fake-psoola.php	test site only	The pretend Psoola page.

"Personal link" means the link contains a secret code made from the person's record and a server secret. It cannot be guessed, and one person's link never opens another person's data.

8 The admin: every page

Page	What you see and can do
Sign in	Email + password, then the 6-digit phone code (Owner and Finance must have it). 5 wrong tries lock the account for 15 minutes.
Dashboard	Big numbers: registered, paid, not paid, students waiting, money confirmed, professionals/students with ticket, emails sent/failed, open sponsor requests. Capacity bars for tickets and lunch. A red "Needs attention" box : students waiting, payments with problems, fake messages, failed emails, missing backup.
Registrations	Search by name, phone, email or reference; filters (status, ticket type, lunch, method, date); possible duplicates marked; export to Excel.
One registration	Every detail, payments, ticket, emails, workshops and history. Buttons: edit contact details (ticket is re-made and re-sent), resend ticket or "Pay now", add workshop, cancel, complimentary ticket (Owner only, with reason). "Register a caller" for phone registrations.
Student approvals	Oldest first, ID photo large, Approve or Reject with a reason.
Payments	Every attempt with Psoola's id, method, amount, status. Problems in red. Daily totals to compare with Psoola's report. Mark refunds. The log of every Psoola message.

Workshops	Seats total / booked / paid / left for each workshop, and who is in it. The website's "seats left" follows these numbers. Export = the trainer's sign-in sheet.
Sponsors & booths	New → Contacted → Agreed → Paid → Confirmed (or Declined / Waiting list). Notes, person in charge. Cannot be Confirmed before Paid; a tier cannot take more than its spots (Owner can override; logged).
Check-in	For the door, works on a phone: scan the QR with the camera, type the ticket number, or search a name. A second scan says "Already checked in at 09:12". Fake or old QR codes are refused.
Settings	Owner only: see section 14. Also ambassador codes (with how many students used each) and the audit log.
Users	Owner only: add people with a role, disable accounts (never deleted, so history stays), reset a lost phone code.
Export	Excel files (Kurdish and Arabic stay correct). Every export is logged with who and when.
Site content	Your existing content admin (texts, speakers, prices) stays as it was.

9 Roles: who can see what

Role	Can use	Phone code
Owner	Everything, including Settings, Users, complimentary tickets and overrides.	required
Registration	Dashboard, registrations (edit), student approvals + photos, workshops, sponsors, check-in, export.	optional
Finance	Dashboard, registrations (view), payments and refunds, sponsors, export.	required
Content	Only the site texts.	optional
Check-in	Only the check-in page (door staff on summit days).	optional

The server checks the role on **every** page and every button, not only by hiding menu items.

10 The database: all 13 tables

You never need to open these directly: the admin shows them as pages. Kurdish and Arabic are stored exactly as typed. Nothing is ever deleted by the system; things are marked (cancelled, disabled, removed) so there is always a history.

Table	What one row is	Seen in admin
registrations	One person: names, phone, email, city, specialty, ticket type, lunch days, payment method, status (waiting / unpaid / paid / complimentary / cancelled), reference, personal-link secrets.	Registrations, Dashboard
student_checks	A student's ID check: university, photo file name, waiting / approved / rejected, reviewer, reason, ambassador code.	Student approvals
payments	One payment attempt: Psoola's id, method, amount expected and confirmed, status (created, waiting, paid, failed, expired, mismatch, duplicate, refunded), Psoola's raw answers.	Payments

tickets	Ticket number, version (a name change makes version 2 and the old QR stops working), how it was made (payment or complimentary), check-in time.	One registration, Check-in
emails	Every email as a queue: kind, to, language, sent / failed, tries.	One registration, Dashboard
workshop_bookings	A person booked into a workshop by phone, paid or not.	Workshops
sponsor_requests	A sponsor or booth request with its status, notes and person in charge.	Sponsors & booths
ambassadors	Ambassador codes, to count students per ambassador.	Settings
webhook_log	Every message Psoola sent and what we did with it.	Payments
admin_users	People who can sign in: role, password (scrambled, never readable), phone-code secret, lock status.	Users
audit_log	Who did what and when. Never edited, never deleted.	Settings
settings	Open/closed, capacity, test mode... (section 14).	Settings
rate_limits	Counters that stop spam and password guessing.	(internal)

11 The emails

All emails are written in the person's language (English, Arabic or Kurdish), in the iSmile colours, with a plain-text copy for old phones. They go out through a queue: if sending fails, it retries; you see failures on the dashboard.

Email	When
We are checking your student ID	A student registers.
Approved: pay now	A student is approved (contains the personal pay link).
Not approved	A student is rejected: the reason + link to send a new photo.
Pay now	A payment failed or expired (max one a day).
Reminder	3 days after registering, still unpaid. Only once.
Sorry, we are full	Capacity reached: unpaid people are told once.
Your ticket	Paid (or complimentary): QR in the email + PDF ticket attached. Again if details change.
Sponsor: received	To the company, with their reference.
Sponsor: new request	To your sponsor team address (set in Settings).
Alert (English, to you)	Wrong amount, paid twice, over capacity, fake messages, many failed emails, missing backup, disk nearly full.

Test mode (on by default): every email goes to one test address instead of the real person, until you switch it off at launch. Sending uses **Brevo** from tickets@ismile.krd; until a Brevo account exists, emails are saved as files instead.

12 The PDF ticket and the QR code

- The ticket is a one-page A4 PDF in the person's language: name, ticket type, lunch days, reference, ticket number and a large QR code.
- The QR code contains the ticket number plus a **signature** made with the server's secret. At the door, the check-in page verifies the signature: a QR made by someone else is refused as **forged**.
- If a name is changed, the ticket becomes version 2 and is emailed again; the **old QR stops working**.
- A second scan shows "Already checked in at ...", so a copied ticket cannot bring two people in.

13 Timed jobs (automatic work)

How often	What it does
Every minute	Sends the queued emails.
Every 5 minutes	Asks Psoola about every payment still waiting (catches lost messages); closes attempts older than 24 hours; sends the 3-day reminders and the "full" emails; checks for problems and sends alerts.
Every night 02:30	Full database backup (kept in the private folder); deletes student ID photos 90 days after the summit (the approve/reject decision stays); cleans old counters.

14 Settings you control (Owner)

Setting	What it does	Starts as
Registration is open	The main switch. Off = form closed and no payments start (emergency stop).	off
Message while closed (EN/AR/KU)	Your own text, e.g. "Registration opens on 25 October".	empty (default text)
Ticket capacity	Registration closes itself at this number.	0 = no limit
Lunch capacity day 1 / day 2	That lunch day disappears from the form when full.	0 = no limit
Email test mode + address	All emails to one test address.	on
Sponsor requests go to	Who gets "new sponsor request".	empty
"Pay now" link valid for	Days a personal pay link works (1–30).	7 days
Prices	Not here: in Site content → Ticket prices (professional, student, lunch day 1, lunch day 2, IQD).	all 0

Secret things (database password, Psoola keys, Brevo key, the server secret) are in `config.php` on the server only. They never appear in the admin, in Git, or in any email.

15 Safety and security

Money

- No ticket without Psoola itself confirming paid + exact amount + currency.

Admin

- Own account per person, 5 roles, checked on the server.

- The server sets the price; the page cannot change it.
- Each Psoola payment can be confirmed only once (the database locks the row while deciding).
- Price 0 cannot be paid.
- Fake webhooks refused and logged.
- Phone code for Owner and Finance; lock after 5 wrong tries.
- Sign-out after 30 minutes idle, 12 hours at most.
- Every button has a one-time token (stops trick links clicking for you).

Visitors' data

- Personal links cannot be guessed and expire.
- ID photos: checked, cleaned, private, deleted after the summit.
- Everything typed is stored as plain text: code or "SQL" in a name does nothing.
- Spam trap + limits on how often one address can send forms.
- Forms posted from other websites are refused.

- Admin pages cannot be shown inside another website.
- Every action and every export in the audit log.

Server

- The backend folder sits outside the public website.
- Always HTTPS (padlock); internal files blocked even if uploaded by mistake.
- Nightly backup; alerts if it is missing.

16 Putting it on the server

- 1 **Two copies: test.ismile.krd** (password-protected, pretend Psoola, emails in test mode) and **ismile.krd** (the real one). Always test first.
- 2 **deploy.sh**: one command puts the project on the server: the website into the public folder, the backend outside it. cPanel's "Deploy" button runs it for the test site (`.cpanel.yml`).
- 3 **merge-data.php**: an update **never overwrites** what you changed in the admin (prices, speakers, texts). It only adds new files and new translation texts.
- 4 **install.php**: creates the 13 tables and your first Owner account.
- 5 **.htaccess** files: HTTPS always, no "www", backend/docs/tools blocked, admin pages never cached.
- 6 **Cron jobs** in cPanel for the timed jobs (section 13).
- 7 Run the 88 checks on the test copy on the real server, then go live.

The step-by-step commands are in `backend/README.md` . I tested the deploy on a pretend server folder on this laptop: 14 new files and 454 new texts were added, and a price changed in the admin was kept.

17 How it was tested (all 88 checks)

A test program (`backend/tests/scenarios.php`) acts like a visitor, like Psoola and like a hacker against a test copy, then looks in the database. It refuses to run on the live site. Result today: **88 passed, 0 failed**. Items marked NEW were added after the security review.

Opening rules

- ✓ prices are set on the test site
- ✓ closed: page told "closed" with the Kurdish message
- ✓ closed: a registration sent anyway is refused
- ✓ open: page told "open"

Professional pays with a card

- ✓ saved as unpaid with a server-made reference
- ✓ price worked out by the server (ticket + lunch)
- ✓ visitor sent to the payment page

- ✓ chosen method recorded
- ✓ webhook → server asks → paid
- ✓ ticket made with a number
- ✓ ticket email queued
- ✓ status page shows paid, ticket number and QR
- ✓ QR image served on the personal link

Webhooks

- ✓ same webhook twice changes nothing
- ✓ still exactly one ticket

- ✓ forged webhook rejected (401)
- ✓ ...and logged as REJECTED
- ✓ genuine webhook about unknown payment changes nothing

Money problems

- ✓ smaller amount NOT paid (mismatch)
- ✓ ...no ticket made
- ✓ ...Finance alerted
- ✓ failed: stays unpaid, no ticket
- ✓ failed: one "Pay now" email
- ✓ "Pay now" link starts a fresh payment
- ✓ wrong "Pay now" token refused
- ✓ expired "Pay now" link refused
- ✓ webhook lost: still waiting
- ✓ 5-minute job finds it paid; ticket made
- ✓ paid after "failed": money not lost **NEW**
- ✓ many failed tries: one email a day **NEW**
- ✓ registration off: pay link cannot start a payment **NEW**
- ✓ "Pay" again within minutes reuses the attempt **NEW**
- ✓ paid twice: second = duplicate, one ticket

Students

- ✓ renamed program as ID photo refused
- ✓ student without photo refused
- ✓ "dental student" always gets student ticket
- ✓ saved as waiting, no payment
- ✓ photo private under a random name
- ✓ photo has no public address
- ✓ admin photo page needs sign-in
- ✓ "checking your ID" email
- ✓ rejected: reason emailed
- ✓ better photo with link → waiting again
- ✓ another person's token cannot upload
- ✓ rejected student cannot be approved before a new photo **NEW**
- ✓ approved: unpaid + "Pay now" email
- ✓ pays student price, gets ticket

Personal links

- ✓ status with someone else's token refused
- ✓ QR with someone else's token refused
- ✓ paid registration cannot pay again

Tickets at the door

- ✓ genuine QR accepted
- ✓ forged QR refused
- ✓ name change re-issues and re-emails
- ✓ old QR stops working

Capacity

- ✓ lunch day full disappears, other day works
- ✓ full lunch day refused
- ✓ tickets full: form closes

Also tested in a real browser (Microsoft Edge, automatically): professional pays in English; student with ID photo in Arabic; failed payment and try again; sponsor form; admin sign-in with phone code and every admin page, approve, add workshop, check-in scan twice, Excel export. No errors.

- ✓ registration sent anyway refused
- ✓ last place held for the person paying **NEW**
- ✓ ...and they get the ticket **NEW**

Typed-in text stays text

- ✓ code in a name saved as text
- ✓ "SQL" in a field did nothing
- ✓ email shows it safely
- ✓ injection in admin search finds nothing

Admin protection

- ✓ sign-in without its token refused
- ✓ 5 admin pages need a sign-in
- ✓ sign-in never sends you to another site (3 tricks) **NEW**
- ✓ odd links do not crash a page **NEW**
- ✓ admin cannot be framed

Sponsors, workshops, jobs, limits

- ✓ sponsor saved with a reference
- ✓ company and team emailed
- ✓ cannot be Confirmed before Paid
- ✓ form from another website refused
- ✓ workshop lowers "seats left"
- ✓ removing frees the seat
- ✓ queued emails sent
- ✓ none left pending
- ✓ 3 days unpaid: one reminder
- ✓ never a second reminder
- ✓ backup written
- ✓ backup contains registrations
- ✓ many forms from one address blocked (429)

18 The security review and the fixes

After building, a second independent check read all the code looking for ways to break it. It confirmed the main rules hold (tickets only after confirmed money, locks against double payment, private photos, prepared database queries, escaped output, protected exports). It found 6 real problems. All are fixed and each now has a check that proves it:

#	Problem	Fix
1	A trick sign-in link could send an admin to a fake site after signing in.	Only our own admin pages are accepted as the next page.
2	Paid after a declined card → money taken, no ticket.	A late "paid" is always checked with Psoola and honoured.
3	A failing link could send "Pay now" emails endlessly.	Max one a day; recent attempts reused.
4	"Registration off" did not stop pay links.	Off now stops every payment.
5	Last places could be oversold when many pay at once.	Places held while paying; Owner alerted if a limit is ever passed.
6	A rejected student could be approved without a new photo.	Only a waiting photo can be approved or rejected.
+	Smaller items.	Pretend Psoola refused on any non-test address; odd links no longer crash pages; photos over 16 megapixels refused; very large page numbers handled.

19 What is not finished yet

Item	Why / what is needed	Waiting on
Psoola connection (A, B, C)	Three marked places in <code>PsoolaGateway.php</code> . About 2 days of work once their documentation and test keys arrive.	Psoola
Server + domain	Set up <code>test.ismile.krd</code> , database, cron jobs, run the 88 checks there.	you (tonight)
Prices, capacity, people	Typed into the admin; no code needed.	you
Brevo account	For real emails + DNS records so they don't land in spam.	you me
3 go-live security items	(a) prices changeable only by Owner/Finance, or an alert on change; (b) check for Cloudflare so spam limits count each visitor; (c) a used phone code can't be reused, and a password change signs out old sessions.	me
Saved to GitHub	All this work is on this laptop only, not yet committed. It should be saved to Git before anything else.	me, with your OK

20 Every file I made

Folder	Files
api/ (public)	<code>config</code> , <code>register</code> , <code>pay</code> , <code>status</code> , <code>qr</code> , <code>student-photo</code> , <code>sponsor</code> , <code>webhook</code> , <code>fake-psoola</code> , <code>_boot</code> · ~440 lines

admin/ (your pages)	login, two-factor, logout, index (dashboard), registrations, registration, students, photo, payments, workshops, sponsors, checkin, settings, users, export, admin.css, admin.js, .htaccess · ~1,900 lines
backend/src (the brain)	App, Db, Auth, Totp (phone codes), Security, Links (personal links), Registrations, Payments (+ Gateway, FakeGateway, PsoolaGateway), Tickets, Outbox (email queue), EmailTemplates, EmailText (3 languages), Mail (Brevo, Log), IdPhotos, Office (admin actions), Sponsors, Settings, SiteData (prices), RateLimit, Validate, Api, Audit, Backup, Lang, Admin/Page, Admin/RegistrationQuery · ~3,500 lines
backend/database	schema.sql: the 13 tables · 265 lines
backend/cron	run.php: the timed jobs
backend/tools	install.php, deploy.sh, merge-data.php
backend/tests	scenarios.php: the 88 checks · 445 lines
backend/	config.sample.php (settings template), README.md (setup guide), composer files (the TCPDF library)
Website	new: payment.html, student-id.html, js/payment.js, js/student-id.js, css/sections/payment-page.css · changed: register.html, sponsor.html, js/sections/registration.js, js/sections/sponsor-form.js, css/sections/register.css, data/i18n (en, ar, ku) · ~1,000 lines
Server files	.htaccess (root, api, admin, backend), .cpanel.yml, .gitignore additions (secrets never go to Git)

In one sentence: the whole registration, payment, ticket and admin system is built and tested; it needs only the server (tonight), Psoola's documents, and your prices to go live.

Other documents on your Desktop: *Backend built (what I did and what I need)* (short version) · *Backend Plan* · *Questions for Psoola* · *How a payment works*.